

RSVM — Runtime State Visibility Module

Parent Standard: Operational Reality Standard

Category: Governance & Enforcement

Subcategory: Runtime State Visibility

Type: Operational Reality Architecture Module

Version: 1.0

Status: Canonical · Open Module

Effective Date: 13 May 2026

Compatibility: OOF Methodology OS · Operational Reality Standard · Structured Reality Standard · Runtime Integrity Standard · INTEGROS · MTVF · UCL · ORGS · EVIP

Authority: OOF

Protection: MIP — Methodological Intellectual Property

Canonical Language: English

Canonical Definition

Runtime State Visibility Module defines the structural conditions under which a system's actual live runtime state remains identifiable, interpretable, reviewable, and operationally visible strongly enough for governance, validation, audit, and operational reality assessment to remain possible during execution.

A system satisfies RSVM only if:

- actual runtime state remains structurally identifiable
- visibility of live operating conditions does not collapse into symbolic status display alone
- active runtime state can be distinguished from assumed, reported, or legacy state
- visibility remains sufficient to review what condition the system is actually in while operating
- hidden or opaque runtime conditions do not replace governable operational visibility

A system whose real operating state cannot be meaningfully seen, interpreted, or reviewed during live execution does not satisfy RSVM.

Module Function

RSVM defines the visibility layer of operational reality governance.

It ensures that operational reality is not inferred only from documentation, dashboards, status labels, or system declarations, but remains connected to live visibility of actual runtime condition.

The module applies wherever a system must preserve visibility of:

- current operating state
- active execution condition
- live control condition
- permission condition
- escalation state
- runtime boundary condition

- supervision state
- intervention state
- consequence-bearing operational mode

Its function is not merely to display information.

Its function is to make live state visible enough that operational reality can still be governed as reality.

Step 1 — Define the Visibility Object

The organization must define what runtime state must remain visible.

Minimum requirement:

- the visibility object is explicit
- the scope of runtime state visibility is structurally bounded
- undefined visibility targets are excluded from valid runtime visibility logic

The visibility object may include:

- operating mode
 - execution role
 - supervision condition
 - escalation condition
 - permission state
 - boundary condition
 - intervention state
 - active system authority condition
-

Step 2 — Define Runtime State Categories

The system must define which categories of runtime state require visibility.

Minimum requirement:

- runtime state categories are explicit
- visibility does not rely on one generic status layer
- materially different operating conditions remain distinguishable

Runtime state categories may include:

- idle
 - active
 - restricted
 - supervised
 - autonomous
 - escalated
 - degraded
 - emergency
-

- suspended
- invalid

Without defined categories, visibility becomes too vague to support operational reality.

Step 3 — Define Visibility Conditions

The system must define what counts as sufficient visibility of actual runtime state.

Minimum requirement:

- visibility conditions are explicit
- reported state is not automatically treated as actual visible state
- the system can distinguish visible reality from symbolic status representation

This means visibility must remain strong enough to determine:

- what the system is doing
- under what active conditions
- in what operating mode
- with what authority
- under which boundaries
- at what level of escalation or restriction

If those conditions cannot be determined, operational reality cannot remain visible.

Step 4 — Define Opaqueness Detection Logic

The system must define how hidden, ambiguous, delayed, or misleading runtime state conditions are identified.

Minimum requirement:

- opaqueness detection logic is explicit
- runtime state cannot remain operationally consequential while structurally invisible
- the system can detect when state appearance no longer matches state visibility

Opaqueness may include:

- outdated state reporting
- symbolic dashboards disconnected from live condition
- hidden escalation state
- active control state without visibility
- silent boundary change
- runtime authority shift without visible state transition
- ambiguous status collapse across materially different conditions

Without opaqueness detection, systems can remain active while becoming operationally unreadable.

Step 5 — Define Visibility Continuity

The system must define how runtime visibility remains preserved through live change.

Minimum requirement:

- visibility continuity is explicit
- state visibility does not disappear during updates, escalation, delegation, or dynamic execution change
- operationally relevant state transitions remain visible across runtime evolution

This means visibility must survive:

- mode change
- escalation
- restriction
- supervision shift
- cross-system handoff
- active intervention
- runtime anomaly
- recovery or failure transition

A system is not operationally visible if only stable moments remain visible while critical transitions become opaque.

Step 6 — Preserve State Visibility Traceability

The system must preserve traceability of runtime state visibility and state-transition visibility.

Minimum requirement:

- visible state transitions are reviewable
- the basis for determining actual runtime state remains reconstructable
- later audit can determine what state was visible, when it changed, and whether visibility remained sufficient during critical operation

If runtime visibility cannot be reconstructed, then operational reality assessment weakens into post-fact assumption.

Step 7 — Restrict Invalid Runtime Visibility

The system must not be treated as valid if live runtime state remains materially opaque, ambiguously represented, or hidden behind symbolic reporting while governance depends on knowing actual operational condition.

Minimum requirement:

- invalid visibility conditions are identifiable
-

- symbolic state reporting is excluded as sufficient runtime visibility proof
 - systems whose real operating condition cannot remain visibly governable are blocked, narrowed, flagged, or
 - invalidated where operational reality requires actual state visibility
-

Scenario

An AI system operates under live runtime conditions and enters an escalated or expanded authority state, but the visible system representation continues to show a normal operating mode.

Application

RSVM is used to ensure:

- actual runtime state remains distinguishable from displayed symbolic state
 - escalation becomes visible as a real operating condition
 - the system cannot preserve normal-state appearance while operating differently in practice
 - governance remains possible because live state stays identifiable
-

Result

The organization gains a stronger operational reality architecture in which hidden runtime escalation cannot remain invisible behind stable surface status.

Scenario

A distributed operational environment contains multiple services, controllers, and runtime transitions, but no reliable visibility exists into which actual state the live system currently occupies.

Application

RSVM is used to ensure:

- runtime state categories are explicit
 - live operating condition remains visible across transitions
 - hidden ambiguity between declared and actual state is detectable
 - later review can reconstruct when visibility was preserved and when it was lost
-

Result

The environment gains a stronger state-governance architecture in which operational reality remains visible enough to support actual control, validation, and accountability.

Canonical Closing Statement

If actual runtime state cannot remain visible during operation, the system may continue to run, but operational reality can no longer be reliably governed.

Related Documents

→ [Operational Reality Standard \(ORS™\)](#).

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>